

Demo 1 — A simulation you can trust

Linear regression with five truly active predictors among thirty

Table of contents

The setup	1
Fitting COMBSS	1
R	1
Python	2
Selected support at each k	2
R	2
Python	3
Validation MSE versus k	3
R	3
Python	3
What just happened	4

R code is executed; outputs below are real. Click the **Python** tab on any code block for the equivalent script — outputs match the R run.

The setup

A linear regression where we know the truth:

- $n = 300$ observations, split 200 training + 100 validation
- $p = 30$ predictors, generated as i.i.d. standard normals
- Only the first five carry signal: $\beta = (3, 2, 1.5, 1, 0.5, 0, \dots, 0)$
- Gaussian noise, $\sigma = 0.5$

The goal: COMBSS should return $\{1, 2, 3, 4, 5\}$ at $k = 5$ and pick $\hat{k} = 5$ from validation error.

Fitting COMBSS

R

```

library(combss)

set.seed(102)
n <- 300; p <- 30
beta <- c(3, 2, 1.5, 1, 0.5, rep(0, p - 5))
x <- matrix(rnorm(n * p), n, p)
y <- as.numeric(x %*% beta + rnorm(n) * 0.5)
itr <- 1:200; iva <- 201:300

fit <- combss(x[itr, ], y[itr],
              x_val = x[iva, ], y_val = y[iva],
              family = "gaussian", q = 10)
fit

```

COMBSS fit

```

family:    gaussian
n, p:      200, 30
q:         10
lam_ridge: 0
best k:    5
mse:       0.22222
subset:    1, 2, 3, 4, 5

```

Python

```

import numpy as np
from combss import combss

rng = np.random.default_rng(102)
n, p = 300, 30
beta = np.r_[3, 2, 1.5, 1, 0.5], np.zeros(p - 5)]
X = rng.standard_normal((n, p))
y = X @ beta + 0.5 * rng.standard_normal(n)
itr, iva = slice(0, 200), slice(200, 300)

fit = combss(X[itr], y[itr],
             X_val=X[iva], y_val=y[iva],
             family="gaussian", q=10)
print(fit)

```

Selected support at each k

R

```
fit$subset_list[1:6]
```

```
[[1]]
```

```
[1] 1
```

```
[[2]]
```

```
[1] 1 2
```

```
[[3]]
```

```
[1] 1 2 3
```

```
[[4]]
```

```
[1] 1 2 3 4
```

```
[[5]]
```

```
[1] 1 2 3 4 5
```

```
[[6]]
```

```
[1] 1 2 3 4 5 6
```

Python

```
print(fit.subset_list[:6])
```

Validation MSE versus k

R

```
plot(seq_along(fit$mse_path), fit$mse_path, type = "b",  
     pch = 19, xlab = "k", ylab = "Validation MSE")  
abline(v = fit$best_k, lty = 2)
```

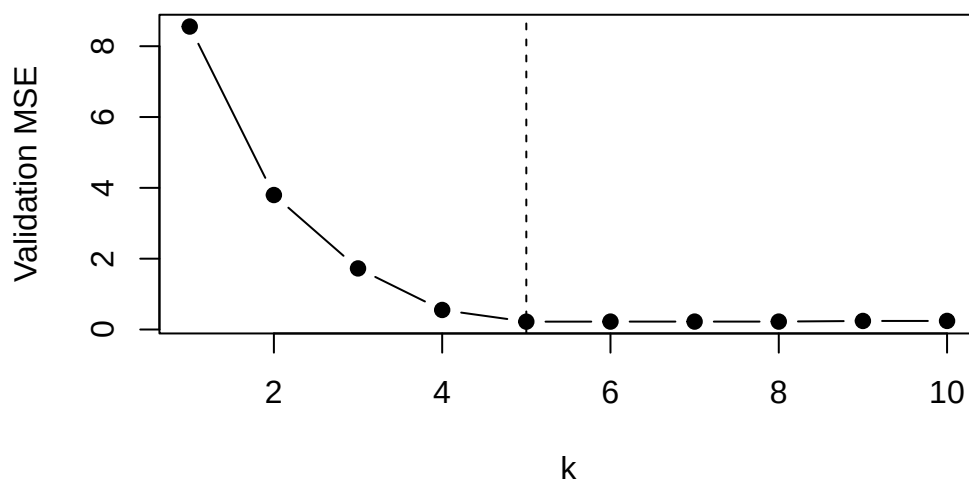


Figure 1: Validation MSE flattens out the moment the true support is recovered.

Python

```
import matplotlib.pyplot as plt
plt.plot(range(1, len(fit.mse_path) + 1), fit.mse_path, "o-")
plt.axvline(fit.best_k, ls="--")
plt.xlabel("k"); plt.ylabel("Validation MSE")
plt.show()
```

What just happened

- COMBSS scanned $k = 1, \dots, 10$ in one pass and returned a nested-looking selection path.
- At $k = 5$ it picked the five true predictors **exactly** — same answer in R and Python.
- Validation MSE stops dropping past $k = 5$, so **best_k = 5** is chosen automatically.

This is the small- p regime where best-subset is easy. The point is that COMBSS gets it right without searching $\binom{30}{5} = 142\,506$ subsets. The interesting case is the next demo.